

StarHarbour — Design Report

1. Engine choice & justification

Choice: a custom event-sourced *decider* engine in Python (the option the brief explicitly permits — “or your own implementation”).

The brief is clear that the deliverable is **workflow design**, not operating infrastructure, and that “idempotency and compensation are still your job — the engine gives you durability, not business rollback.” So I built the smallest thing that forces me to confront exactly those concerns, using the *same model* Temporal and Cadence use internally: an append-only event history plus a pure function that folds it into state.

What this buys us:

- **Determinism and replay are first-class and testable** with no Cassandra, no Docker, no AWS account, no Step Functions Local. A unit test can fold a history and assert the rebuilt state equals the live state (`test_replaying_history_reproduces_state`).
- **F6 (crash recovery) is a real, demonstrable code path**, not a framework feature we trust blindly: process 1 writes events to a JSON-lines log and “crashes”; process 2 constructs a brand-new runtime with a *fresh* payment client, rebuilds state from the log, and continues — with zero re-charges (`test_f6_crash_after_payment_resumes_without_double_charge`).
- **The design transfers directly.** `apply` ↔ Temporal’s event-sourced replay; `decide` ↔ the workflow function’s deterministic command emission; `WorkflowRuntime` ↔ the worker; activities, signals, and timers map 1:1.

HOW IT COMPARES TO THE REAL ENGINES

	This engine	Temporal / Cadence	Step Functions
Workflow expressed as	pure <code>decide</code> / <code>apply</code> functions	fault-oblivious code (Go/Java/...)	declarative ASL JSON
Durability mechanism	event log + fold (event sourcing)	event-sourced history replay	managed service state
Determinism constraint	yes — <code>decide</code> / <code>apply</code> are pure, no IO/clock	yes — replay re-runs your code	n/a (logic pushed to Lambdas)
Signals / timers / queries	events + runtime timers	native	task tokens / Wait states
Compensation	your job (this report §3)	your job	your job
Ops cost	none (in-process)	cluster + DB	managed (AWS)

If this were going to production at scale I'd port the same state machine onto **Temporal** (actively developed successor to Cadence, multi-language, the strongest signal/timer/query story). The `decide` / `apply` split means that port is mechanical: the pure core moves into the workflow function unchanged.

2. The determinism constraint, explained

Durable-execution engines recover by **replaying history** through your workflow logic. For replay to rebuild the *same* state, the workflow logic must be a **deterministic function of recorded events only**. If it read the wall clock, generated a random id, or called a network service directly, a replay could take a different branch than the original run and corrupt state.

This codebase enforces the constraint structurally by splitting responsibilities:

- **Pure / deterministic** (`workflow.py`): `apply(state, event) -> state` and `OrderWorkflow.decide(state) -> Effects`. No clock, no IO, no randomness. Same history $\Rightarrow$ same state $\Rightarrow$ same decisions, every time.
- **Non-deterministic, quarantined** (`engine.py`): the runtime owns the clock, retries with backoff, the network calls, id generation (inside the fake clients), and persistence. Every non-deterministic *result* is captured as an **event** the moment it happens (`PaymentTaken` carries the charge id; `InventoryReserved` carries the reservation id).

Because results are recorded as events, replay never re-derives them — it reads them back. Replay only re-issues activities that have **no recorded result yet**, and those are idempotent (§3). That is the entire recovery story:

Recover = fold the log to rebuild state → ask `decide` what's next → it points at the one activity whose result isn't in the log → re-run it idempotently.

Concretely (F6): if the worker dies *after* `PaymentTaken` is appended, the recovered `decide` sees `status=BREWING` and never calls payment again. If it dies *before* `PaymentTaken` is appended, `decide` sees `status=TAKING_PAYMENT` and re-runs `take_payment` — but the fixed `Idempotency-Key` makes the gateway return the *same* charge, so money still moves exactly once (`test_f6_crash_during_payment_retries_is_idempotent`).

3. Compensation & cancellation policy

The engine gives durability; **business rollback is explicit code**. Two mechanisms:

3.1 IDEMPOTENCY (PREVENT THE NEED TO ROLL BACK)

- **Payments:** every order carries a stable `Idempotency-Key` for its whole life. Charging the same key twice returns the same charge — across retries (F1) and crash-replays (F6). Refunds are likewise idempotent.
- **Activities are safe to re-run:** validate/reserve/notify/loyalty are written so a duplicate invocation is harmless, because retries and recovery can call them more than once.
- **Out-of-context / duplicate signals are no-ops:** `apply` ignores events that don't fit the current status, so a late “collected” or a double “cancel” can't corrupt state (`test_out_of_context_events_are_noops`).

3.2 COMPENSATION (UNDO COMMITTED SIDE EFFECTS)

State tracks what has been committed (`inventory_reserved`, `payment_charged`) and what has been undone (`inventory_released`, `payment_refunded`). Entering `COMPENSATING` sets two fields — `terminal_target` (FAILED/CANCELLED/ABANDONED) and `do_refund` — and the decider then runs the pending undo steps in a fixed order (**refund money** → **release inventory** → **finalize/notify**) before reaching the terminal state. Each step is its own durable event, so compensation is itself crash-safe.

Cancellation behaviour differs before vs. after the drinks are made (a requirement):

- **Before** `READY` (cancel, decline, payment fail, out-of-stock): the order is refundable — `CANCELLED` / `FAILED` with refund (if charged) + inventory release.

- **At/after** `READY` (cancel after the drinks exist, or pickup expiry): the drinks are physically made and paid for, so the **waste policy** applies — `ABANDONED`, **no refund**, inventory reservation released. This is the deliberate F5 / late-cancel decision; a different business might choose a partial refund, which is a one-line change to `do_refund`.

3.3 TIMERS & SLAS

- **Substitution deadline** (F3): armed while `AWAITING_SUBSTITUTION`; firing cancels + refunds.
- **Brew SLA** (barista too slow): armed while waiting for the barista; firing escalates **once** (notify manager / comp) and keeps waiting — the order is not killed, because the customer still wants the drinks.
- **Pickup expiry** (F5): armed while `READY`; firing abandons under the waste policy.

Timers are reconciled declaratively: `decide` returns the set of timers that *should* be armed for the current state, and the runtime arms missing ones and cancels ones no longer wanted — so a state transition (e.g. barista signal) automatically disarms the SLA timer.

4. Known simplifications (honest scope notes)

- **Timer deadlines are in-memory**, re-armed from state on recovery rather than persisted with absolute fire times. A production port (or Temporal) would persist timer fire-times so a crash near a deadline preserves the original deadline. The recovery of *order state* itself is fully durable.
- **Clients are in-process fakes** modelling the specific failure modes the brief calls out (idempotent charge, transient timeout, permanent decline, out-of-stock). Swapping them for real HTTP clients to the hw1 Payments/Loyalty APIs is isolated to `clients.py`.
- **Single-worker, synchronous runtime**. The loop is single-stepped so it is fully controllable from tests; concurrency/leasing across many workers is out of scope (and is exactly what a real engine would provide).